

Advanced Python

Any AI service that calls an API, reads a file, or hits a DB is IO-bound. asyncio turns a single thread into a scheduler for thousands of in-flight tasks — with zero extra infra.

What percentage of your Week-3 API calls are actually waiting on IO?

L1

WHAT'S INSIDE

- 01** Concurrency vs parallelism
- 02** `async def` + `await`
- 03** Pydantic — types at the boundary
- 04** Error handling — at the boundary
- 05** Fan-out demo — 40× speed-up

WHY THIS LESSON MATTERS

5 QPS vs 500 QPS — the gap is async.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Concurrency vs parallelism in one sentence each.

2

Write `async def` + `await` correctly.

3

Fan out N network calls with `asyncio.gather`.

4

Validate API payloads with Pydantic.

5

Report validation errors cleanly with 422.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

Concurrency vs parallelism

One sentence each — memorise

02

async def + await

The vocabulary

03

Pydantic — types at the...

One Pydantic model per endpoint.
Errors auto-route to 422 with field-level detail.

04

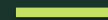
Error handling — at the...

Exceptions flow up; 422 goes out

01

PART 1 OF 4

Concurrency vs parallelism



One sentence each — memorise

Concurrency vs parallelism

Concurrency: many things IN FLIGHT at once.

Parallelism: many things EXECUTING at once.

async = concurrency (single thread, IO yields).

multiprocessing = parallelism (CPU cores).

PICK BY

IO vs CPU

IO-bound → async. CPU-bound → multiprocessing.

DEFINITION

Concurrency vs parallelism

One sentence each — memorise

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

async def + await

async def fetch(url): returns a coroutine.

await runs it; yields on IO.

asyncio.gather(*tasks) fans out in parallel.

Never call blocking code (time.sleep, requests) inside async.

CORE RULE

Never block

Use await asyncio.sleep() and async HTTP clients only.

KEY POINTS

- async def fetch(url): returns a coroutine
- await runs it; yields on IO
- asyncio.gather(*tasks) fans out in parallel
- Never call blocking code (time.sleep, requests) inside async

CONCEPT 3 OF 4

Pydantic — types at the boundary

One Pydantic model per endpoint. Errors auto-route to 422 with field-level detail.

DEFINITION

Pydantic — types at the boundary

One Pydantic model per endpoint. Errors auto-route to 422 with field-level detail.

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 4 OF 4

Error handling — at the boundary

Let Pydantic raise `ValidationError` naturally.

FastAPI maps it to HTTP 422 + structured body.

Never swallow exceptions — log, surface, propagate.

Use tenacity for transient retry logic.

STATUS

422 = validation

Unprocessable Entity. Specific, machine-readable, frontend-friendly.

KEY POINTS

- Let Pydantic raise `ValidationError` naturally
- FastAPI maps it to HTTP 422 + structured body
- Never swallow exceptions — log, surface, propagate
- Use tenacity for transient retry logic

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •
SHARE

ACTIVITY

Measure the 40× speed-up

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1

2

3

4

Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Which Python script of yours most needs asyncio right now?

Q2

What validation error has bitten you that Pydantic would have caught?

Q3

When would you reach for multiprocessing instead?

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

Which is IO-bound?

A

Matrix multiply

B

Parsing 1M JSON lines from disk

C

Image filter

D

Pi calculation

Which is IO-bound?



CORRECT ANSWER

Parsing 1M JSON lines from disk

WHY

IO-bound = disk or network wait dominates; asyncio helps

Inside async code, `time.sleep(3)` causes:

A

Nothing

B

Blocks event loop

C

Error

D

Crash

Inside async code, `time.sleep(3)` causes:

CORRECT ANSWER



Blocks event loop

WHY

It blocks the event loop, starving every other coroutine. Use `await asyncio.sleep`

Pydantic validation failure → HTTP:

A

B

B

D

C

Answer: D. FastAPI maps ValidationError to 422 Unprocessable Entity.

D

(no option)

Pydantic validation failure → HTTP:



CORRECT ANSWER

D

WHY

The correct option matches the lesson's core principle.

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

Concurrency: many things IN FLIGHT at once.

Parallelism: many things EXECUTING at once.

async = concurrency (single thread, IO yields).

multiprocessing = parallelism (CPU cores).

PICK BY

IO vs CPU

IO-bound → async. CPU-bound → multiprocessing.

THE TAKEAWAY

"That is the production Python spine. Everything else is style."

WHAT TO NOTICE

1

Concurrency vs parallelism

2

async def + await

3

Pydantic — types at the...

4

Error handling — at the...

Mini assignment — FastAPI + asyncio + Pydantic

Rewrite your W3 P2 Python microservice using FastAPI + asyncio + Pydantic.

DELIVERABLES

- Replace Flask with FastAPI.
- Define a Pydantic HouseInput model.
- Convert /predict to async def.
- Measure QPS before and after.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Repo URL + before/after QPS numbers on the forum.

ONE THING TO REMEMBER

“

"That is the production Python spine. Everything else is style."

— AI Learning Hub

END OF LESSON 4

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Lesson 2 — Computer Vision. Your first face-detection pipeline.